

Call(), apply() and bind()

The Problem

```
const user = {
  name: "Satyam",
  greet : function() {
    console.log(this.name)
  }
}
const newFun = user.greet
//So, it loses the object context
newFun()
```

call() -

→ call() allows us to manually set this

Syntax →

function.call (obj, arg1, arg2, ...)

↑ individually comma separated

... ^ /

↓
this Arguments.

→ call executes the function immediately

```
//call() methods
const user = {
  name : "Satyam"
}

function greet(message,age){
  console.log(`${this.name} ${message}`)
  console.log("Aapka age: ",age)
}

greet.call(user, "Kese ho",25 )
```

apply()

→ apply () works almost exactly like call ()

→ arguments in an array -

→ immediate ~~call~~ exefutos.

Syntax

function. apply (object , [arg1 , arg2])

↓

this argu

```
//apply()
```

```
const user = {
  name : "Satyam"
}
function greet(message,age){
  console.log(`${this.name} ${message}`)
  console.log("Aapka age: ",age)
}
greet.apply(user, ["Kese ho",25] )
```

bind() .

- > It does not execute the function immediately.
- + it returns a new function with this fixed to the specified object

Syntax

`function.bind (object , arg1 , arg2 ... )`

```
// bind() method
const user = {
  name : "Satyam"
}
function greet(){
  console.log(this.name)
}
const boundGreet = greet.bind(user)
boundGreet()
```

//pre filled arguments or multiple arguments

```

const user = {
  name : "Satyam"
}
function greet(age,city){
  console.log(this.name,age,city)
}
// const boundGreet = greet.bind(user, 25 ,"Noida")
// boundGreet()
const boundGreet = greet.bind(user, 25)
boundGreet("Patna")

```

why use bind()

```

// why use bind()
const user = {
  name : "Rahul",
  greet : function (){
    console.log(this.name)
  }
}
const newFun = user.greet.bind(user)
setTimeout(newFun, 1000)

```

# method	set this	executes immediately	argument
call()	→ yes	→ yes	→ Individuals (comma seprate)
apply	→ yes	→ yes	→ <u>Array</u>
bind()	→ yes	→ No	→ Individual (pre-filled)